1. Create a sorting hat program called sorting_hat that, when run, takes in Y or N answers and returns your Hogwart's house between Gryffindor and Slytherine using if/elif/else statements. The sorting hat program should have at least 2 simple yes/no questions using python's input function. You can use any questions that you'd like and determine the Hogwart's house any way that you'd like. Have the selected house be the return value of the function.

In [95]:
```
def sorting_hat():
    q1 = input("Do you value your own safety above all else? (Y/N): ").strip().uppe

    q2 = input("Are you sure? (Y/N): ").strip().upper()

    if q1 == 'Y' and q2 == "N":
        HOUSE = "Gryffindor!"
    elif q1 == 'N' and q2 == "Y": #ANY PERSON ANSWERING BOTH YES & NO IN EITHER ORD
        HOUSE = "Gryffindor!"
    elif q1 == 'Y' and q2 == "Y":
        HOUSE = "Slytherine!"
    elif q1 == 'N' and q2 == "N": #ANY PERSON ANSWERING ONLY YES OR NO WILL BE SLYT
        HOUSE = "Slytherine!"
    # USED GEMINI TO HELP WITH CODE ERROR
    print(HOUSE)

sorting_hat()
```

```
Do you value your own safety above all else? (Y/N): N
Are you sure? (Y/N): Y
Gryffindor!
```

2. Write a class mydata that takes in a list of numbers and has the methods: mean and std (for standard deviation) returning the mean and standard deviation respectively. This should calculate the mean and standard deviation using only base python functions. Try it out with x = mydata([1, 2, 7, 5]).

In [96]:
```
#wrote the base of this code myself and applied GEMINI corrections
class mydata: #takes in a list
  def __init__(self, input_list):
    self.my_list = input_list

  def mean(self): #calculates the mean of the list
    return sum(self.my_list) / len(self.my_list)

  def std(self): #calculates the sd of the list
    n = len(self.my_list)
    data_mean = self.mean()
    variance = sum((x - data_mean) ** 2 for x in self.my_list) / (n - 1) # Sample s
    return variance ** 0.5

  def print(self): #new method that prints both the mean and sd
    return(self.mean(), self.std())
```

```
x = mydata([1, 2, 7, 5])
x.print()
```

Out[96]: (3.75, 2.753785273643051)

3. Create a function called lag as follows. It takes in a numpy vector, say a = numpy.array([1, 6, 8, 4]) and an integer > 0. It then returns the array lagged by the integer value. So, lag(a, 1) will return [nan, 1, 6, 8] and lag(a, 2) will return [nan, nan, 1, 6]. That is, it gets rid of n values at the end and appends that many nans at the beginning. Use numpy's nan for the missing values. It should return a numpy array. Do not use numpy's existing lag, lead, roll etcetera functions. Instead, use numpy indexing.

In [97]:
```
#my original code was completely off, GEMINI hated it a lot
#i applied the chnages but kept my original sytax of creating an empty array and ad
import numpy as np

def lag(array, num_lags):
  #create an empty array and add "nan"*num_lags
  nans = np.array([])
  nans = np.append(nans, [np.nan] * num_lags)

  #wanted values are kept by removing the indexed values given by -num_lags
  kept_values = array[:-num_lags]

  # Concatenate the NaNs with the remaining part of the array
  return np.concatenate((nans, kept_values))

a = np.array( [1, 6, 8, 4] )
print(lag(a, 2))
print(lag(a, 1))
```

```
[nan nan  1.  6.]
[nan  1.  6.  8.]
```

4. Using your code from 3, write a function lagGenerator that returns - a function - that lags by a specific amount. So lagGenerator(2) creates a function that lags a vector by 2. Use the lambda operator in python.

In [98]:
```
#i was using lambda wrong by adding a lot fo lines when it can only be a simple com
#cannot use "return" or try to make new arrays within a lambda function
import numpy as np

lagGenerator = lambda n: lambda x: np.concatenate((np.full(n, np.nan), x[:-n]))

a = np.array( [1, 6, 8, 4] )
fn = lagGenerator(1)
print(fn(a))
```

```
[nan  1.  6.  8.]
```

5. Create a function called filter_and_square. This function should take a list of numbers as input. Using a list comprehension, return a new list that contains the squares of the numbers, but only for the numbers in the original list that are greater than 0. For example, [-2, 0, 3, 4] should return [9, 16].

In [99]:
```
def filter_and_square(input_list):
  new = [x for x in input_list if x > 0] #new list that only includes filtered valu
  return [new**2 for new in new] #squares the individual values in the list

print(filter_and_square([-2, 0, 3, 4, 8, 0, 16, -4]))
filter_and_square([-2, 0, 3, 4])
```

```
[9, 16, 64, 256]
```

Out[99]:
```
[9, 16]
```

6. Create a function called frequency_map. This function takes a list of strings (e.g., ['apple', 'banana', 'apple', 'orange']) and returns a dictionary where the keys are the strings and the values are the counts of how many times each string appears in the list. Do not use collections.Counter; write the logic using basic dictionary operations.

In [100…
```
def frequency_map(input_list):
  frequency_map = {x: input_list.count(x) for x in set(input_list)} #set() returns
  return frequency_map

list = ['apple', 'banana', 'apple', 'orange', 'orange', 'watermelon', "pear", 'pear
frequency_map(list)
```

Out[100…
```
{'apple': 2, 'banana': 1, 'pear': 3, 'watermelon': 1, 'orange': 2}
```

7. Create a class called extended_data that inherits from your mydata class in question 2. Add two new methods to this class: min (returns the minimum value in the data) and max (returns the maximum value). Ensure that mean and std still work for instances of this new class.

In [101…
```
#Gemini didnt like my manual way of finding the min/max by taking the index 0 of a
#sorted list (reverse=False/True respectively) and returning that value for the sel
class extended_data(mydata):
  def __init__(self, input_data):
    # input_data gets checked somehow ?
    if isinstance(input_data, mydata):
      super().__init__(input_data.my_list)
    else:
      super().__init__(input_data)

  def min(self): #calculates the min of the list
    return min(self.my_list)

  def max(self): #calculates the max of the list
```

```
    return max(self.my_list)

  def print(self): #new method that prints min and max in addition to the original
    return(self.mean(), self.std(), self.min(), self.max())


x = mydata([1, 2, 7, 5])
y = extended_data(x)
y.print()
```

Out[101…

```
(3.75, 2.753785273643051, 1, 7)
```